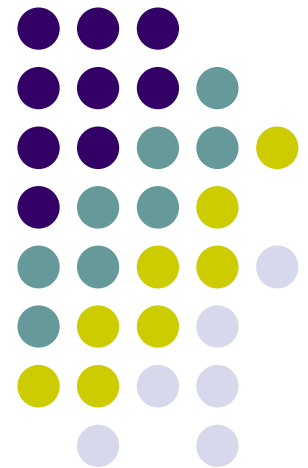
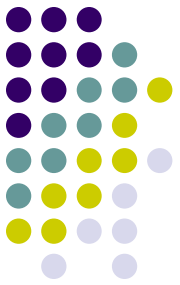
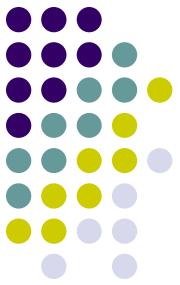


第5章 回溯法





- 学习要点
- 理解回溯法的深度优先搜索策略。
- 掌握用回溯法解题的算法框架
- (1) 递归回溯
- (2) 迭代回溯
- (3) 子集树算法框架
- (4) 排列树算法框架



- 通过应用范例学习回溯法的设计策略。
- (1) 装载问题;
- (2) 批处理作业调度;
- (3) 符号三角形问题
- (4) n 后问题;
- (5) 0-1背包问题;
- (6) 最大团问题;
- (7) 图的 m 着色问题
- (8) 旅行售货员问题
- (9) 圆排列问题
- (10) 电路板排列问题
- (11) 连续邮资问题



● 搜索算法介绍

(1) 穷举搜索

(2) 盲目搜索

- 深度优先(DFS)或回溯搜索(Backtracking);
- 广度优先搜索(BFS);
- 分支限界法(Branch & Bound);
- 博弈树搜索($\alpha - \beta$ Search)

(3) 启发式搜索

- A* 算法和最佳优先(Best-First Search)
- 迭代加深的A*算法
- B*, AO*, SSS*等算法
- Local Search, GA等算法



● 搜索空间的三种表示：

- 表序表示：搜索对象用线性表数据结构表示；
- 显式图表示：搜索对象在搜索前就用图（树）的数据结构表示；
- 隐式图表示：除了初始结点，其他结点在搜索过程中动态生成。缘于搜索空间大，难以全部存储。

● 搜索效率的思考：随机搜索

- 上世纪70年代中期开始，国外一些学者致力于研究随机搜索求解困难的组合问题，将随机过程引入搜索；
- 选择规则是随机地从可选结点中取一个，从而可以从统计角度分析搜索的平均性能；
- 随机搜索的一个成功例子是：判定一个很大的数是不是素数，获得了第一个多项式时间的算法。



回溯法

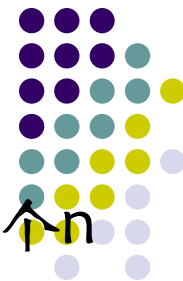
回溯法是一个既带有系统性又带有跳跃性的搜索算法；

它在包含问题的所有解的解空间树中，按照深度优先的策略，从根结点出发搜索解空间树。—— **系统性**

算法搜索至解空间树的任一结点时，判断该结点为根的子树是否包含问题的解，如果肯定不包含，则跳过以该结点为根的子树的搜索，逐层向其祖先结点回溯。否则，进入该子树，继续深度优先的策略进行搜索。—— **跳跃性**

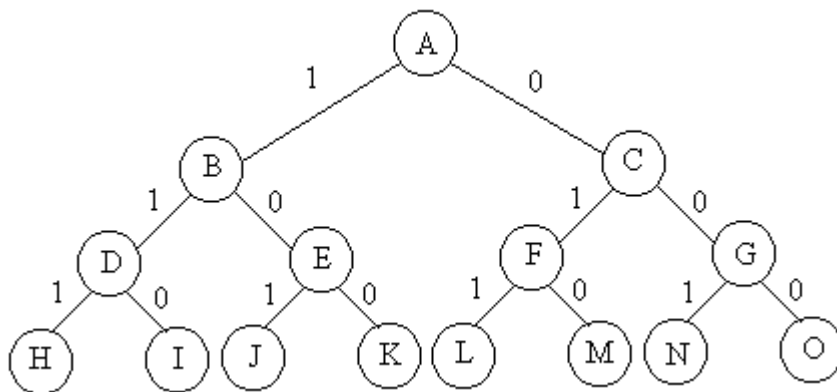
这种以深度优先的方式系统地搜索问题的解得算法称为回溯法，它适用于解一些组合数较大的问题。

问题的解空间



- 问题的解向量：回溯法希望一个问题的解能够表示成一个n元式 $(x_1, x_2, \dots, x_n)$ 的形式。
- 显约束：对分量 x_i 的取值限定。
- 隐约束：为满足问题的解而对不同分量之间施加的约束。
- 解空间：对于问题的一个实例，解向量满足显式约束条件的所有多元组，构成了该实例的一个解空间。

注意：同一个问题可以有多种表示，有些表示方法更简单，所需表示的状态空间更小（存储量少，搜索方法简单）。

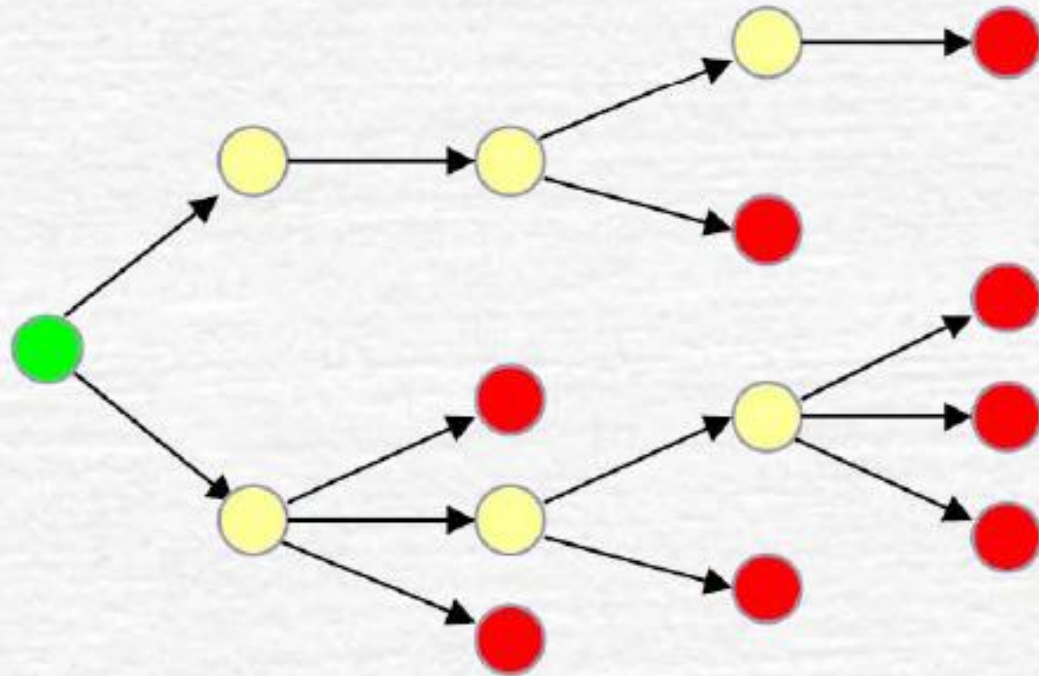


n=3时的0-1背包问题用完全二叉树表示的解空间



通常将解空间组织成树或图的形式

解空间树：



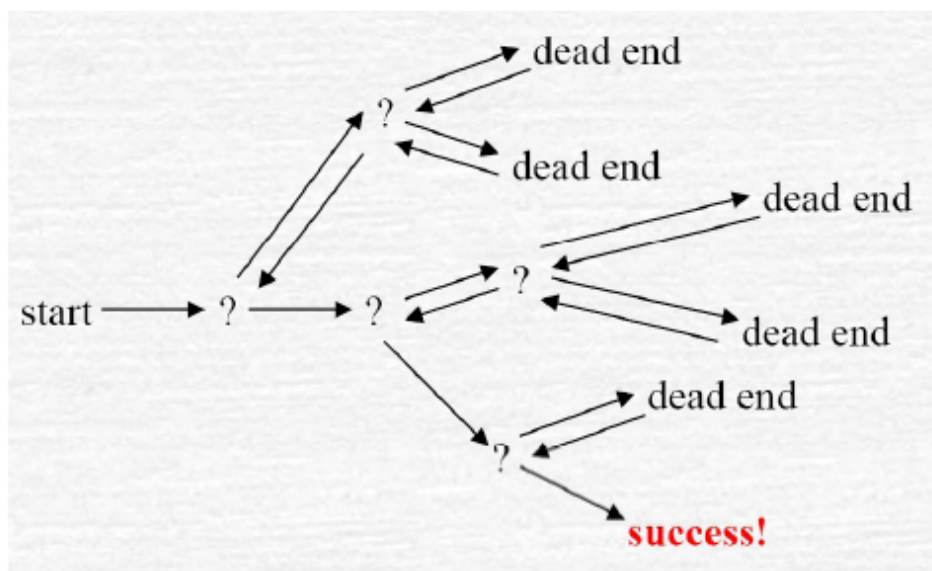
有三种结点：

- 根结点（搜索的起点）
- 中间结点（非终端结点）
- 叶结点（终端结点）
，叶结点为解向量

搜索过程就是找一个或一些特别的叶结点。



- 搜索从开始结点（根结点）出发，以深度优先搜索整个解空间。
- 这个开始结点成为活结点，同时也成为当前的扩展结点。在当前的扩展结点处，搜索向纵深方向移至一个新结点。这个新结点就成为新的活结点，并成为当前扩展结点。
- 如果在当前的扩展结点处不能再向纵深方向扩展，则当前扩展结点就成为死结点。
- 此时，应往回移动（回溯）至最近的一个活结点处，并使这个活结点成为当前的扩展结点；直到找到一个解或全部解。



生成问题状态的基本方法



- 扩展结点:一个正在产生儿子的结点称为扩展结点
- 活结点:一个自身已生成但其儿子还没有全部生成的节点称做活结点
- 死结点:一个所有儿子已经产生的结点称做死结点
- 深度优先的问题状态生成法: 如果对一个扩展结点R, 一旦产生了它的一个儿子C, 就把C当做新的扩展结点。在完成对子树C (以C为根的子树) 的穷尽搜索之后, 将R重新变成扩展结点, 继续生成R的下一个儿子 (如果存在)
- 宽度优先的问题状态生成法: 在一个扩展结点变成死结点之前, 它一直是扩展结点
- 回溯法: 为了避免生成那些不可能产生最佳解的问题状态, 要不断地利用限界函数(bounding function)来处死那些实际上不可能产生所需解的活结点, 以减少问题的计算量。具有限界函数的深度优先生成法称为回溯法

回溯法的基本思想



- (1) 针对所给问题，定义问题的解空间；
- (2) 确定易于搜索的解空间结构；
- (3) 以深度优先方式搜索解空间，并在搜索过程中用剪枝函数避免无效搜索。

常用剪枝函数：

用约束函数在扩展结点处剪去不满足约束的子树；

用限界函数剪去得不到最优解的子树。

递归回溯



回溯法对解空间作深度优先搜索，因此，在一般情况下用递归方法实现回溯法。

```
void Backtrack (int t) // t表示递归深度
{
    if (t>n) Output(x);
    else // f(n, t)和g(n, t)表示未搜索子树的起止编号
        for (i=f(n, t); i<=g(n, t); i++) {
            x[t]=h(i); // 当前扩展结点x[t]的第i个可选值
            if(Constraint(t)&&Bound(t)) Backtrack(t+1);
        } // Constraint() 约束函数, Bound() 界限函数
    } // 算法结束
```

迭代回溯



采用树的非递归深度优先遍历算法，可将回溯法表示为一个非递归迭代过程。

```
void iterativeBacktrack ()
{
    int t=1;
    while (t>0) {
        if (f(n,t)<=g(n,t))
            for (int i=f(n,t);i<=g(n,t);i++) {
                x[t]=h(i);
                if (constraint(t)&&bound(t)) {
                    if (solution(t)) output(x);
                    else t++;} //Solution(t)判断在当前扩展结点处是否得到可行解
            }
        else t--;
    }
}
```



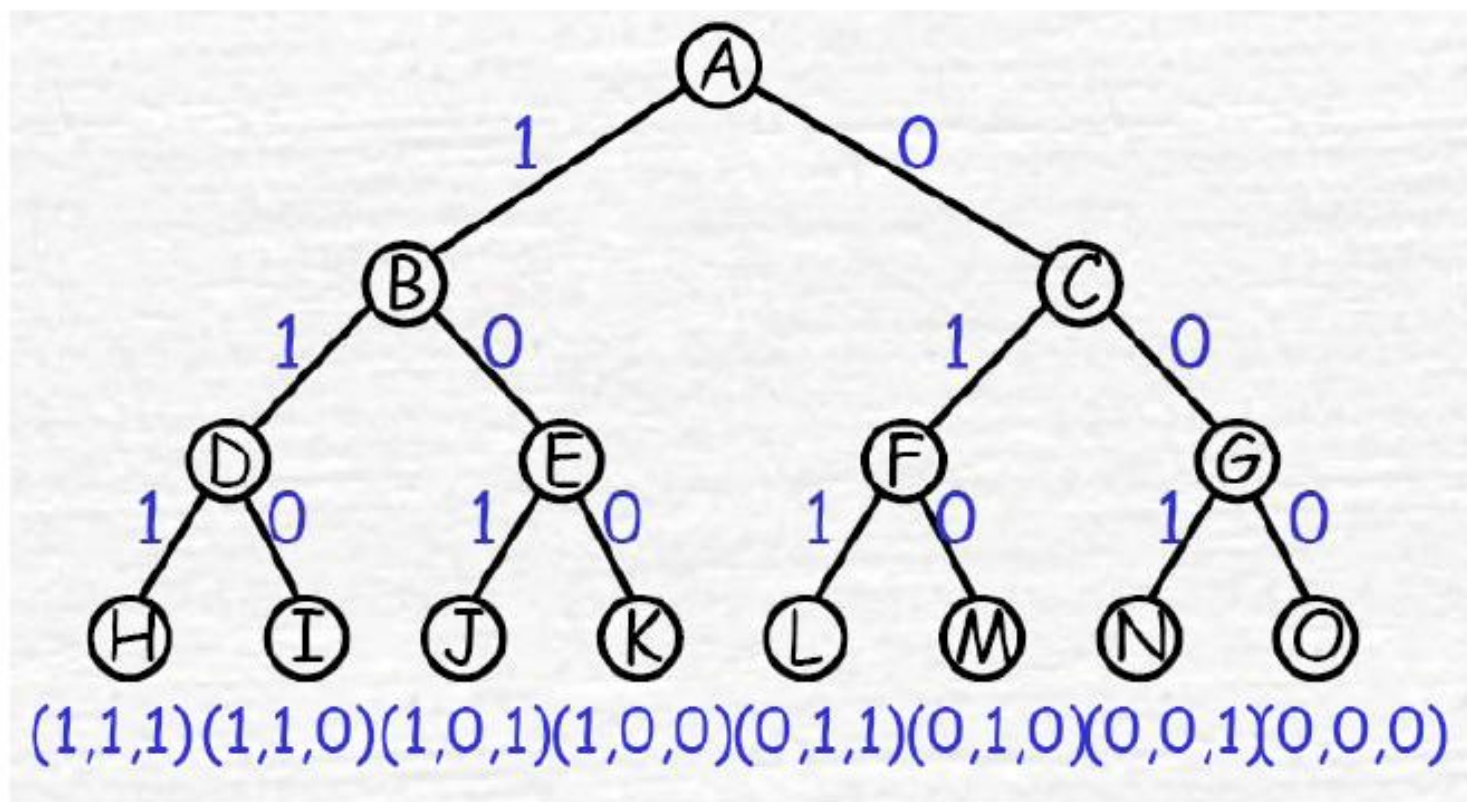
用回溯法解题的一个显著特征是在搜索过程中动态产生问题的解空间。在任何时刻，算法只保存从根结点到当前扩展结点的路径。如果解空间树中从根结点到叶结点的最长路径的长度为 $h(n)$ ，则回溯法所需的计算空间通常为 $O(h(n))$ 。而显式地存储整个解空间则需要 $O(2^{h(n)})$ 或 $O(h(n)!)$ 内存空间。



例1 [0-1背包]: $n=3$, $w=(16, 15, 15)$, $v=(45, 25, 25)$, $c=30$

(1) 定义解空间: $X=\{(0,0,0), (0,0,1), (0,1,0), \dots, (1,1,0), (1,1,1)\}$

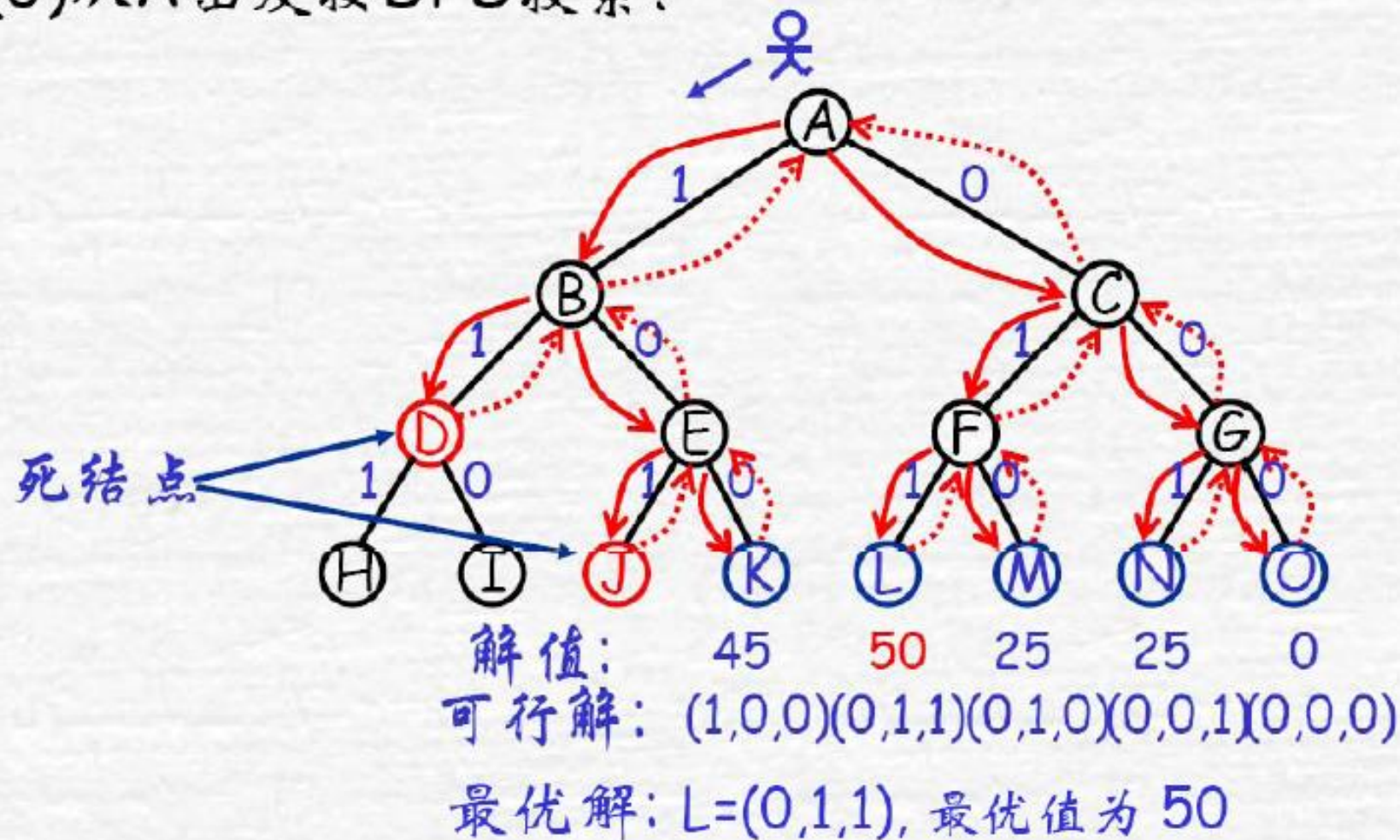
(2) 构造解空间树:



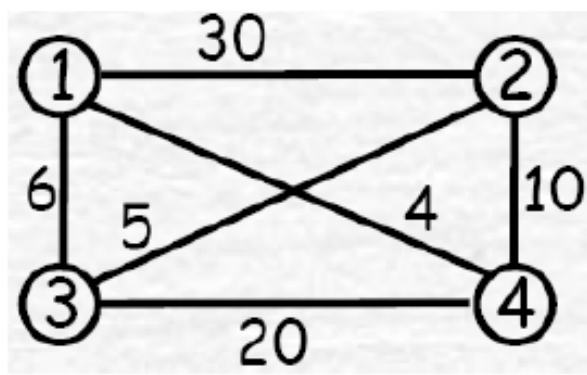


● 例1(cont.): $n=3$, $w=(16,15,15)$, $v=(45,25,25)$, $c=30$

(3)从A出发按DFS搜索:



● 例2 [TSP问题]:



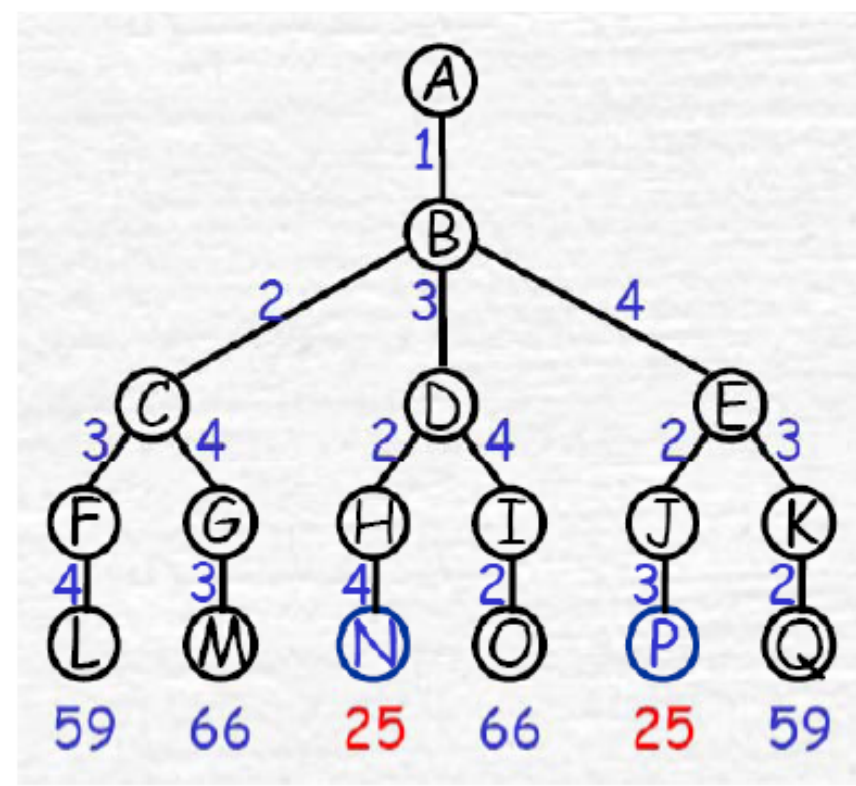
(1) 定义解空间: $X = \{12341, 12431, 13241, 13421, 14231, 14321\}$

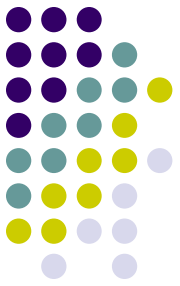
(2) 构造解空间树:

(3) 从A出发按DFS搜索整棵树:

最优解: 13241, 14231

成本: 25





常用剪枝函数

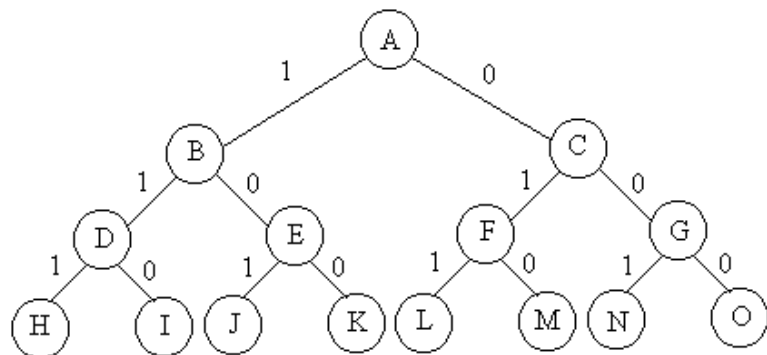
- 用约束函数在扩展结点处剪去不满足约束的子树
- 用限界函数剪去得不到最优解的子树。
- 0-1 背包问题?
- TSP 问题?



● 二类常见的解空间树：

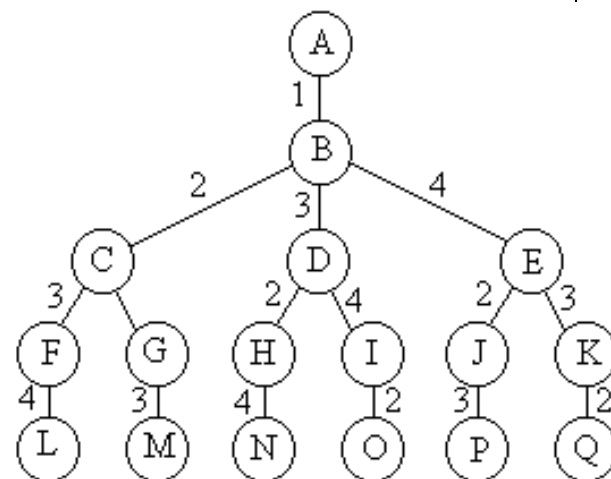
- ① **子集树**：当所给的问题是从 n 个元素的集合 S 中找出满足某种性质的子集时，相应的解空间树称为子集树。子集树通常有 2^n 个叶子结点，其总结点个数为 $2^{n+1}-1$ ，遍历子集树时间为 $\Omega(2^n)$ 。如0-1背包问题，叶结点数为 2^n ，总结点数 2^{n+1} ；
- ② **排列树**：当所给问题是确定 n 个元素满足某种性质的排列时，相应的解空间树称为排列树。排列树通常有 $n!$ 个叶子结点，因此，遍历排列树需要 $\Omega(n!)$ 的计算时间。如TSP问题，叶结点数为 $n!$ ，遍历时间为 $\Omega(n!)$ 。

子集树与排列树



遍历子集树需 $O(2^n)$ 计算时间

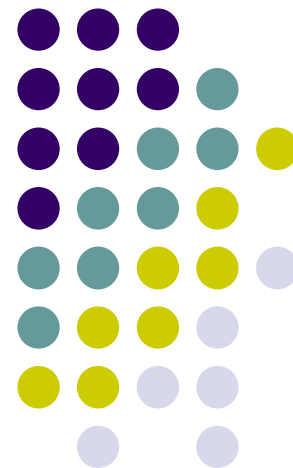
```
void backtrack (int t)
{
    if (t>n) output(x);
    else
        for (int i=0;i<=1;i++) {
            x[t]=i;
            if (legal(t)) backtrack(t+1);
        }
}
```



遍历排列树需要 $O(n!)$ 计算时间

```
void backtrack (int t)
{
    if (t>n) output(x);
    else
        for (int i=t;i<=n;i++) {
            swap(x[t], x[i]);
            if (legal(t)) backtrack(t+1);
            swap(x[t], x[i]);
        }
}
```

装载问题



装载问题



有一批共 n 个集装箱要装上2艘载重量分别为 c_1 和 c_2 的轮船，其中集装箱 i 的重量为 w_i ，且 $\sum_{i=1}^n w_i \leq c_1 + c_2$

装载问题要求确定是否有一个合理的装载方案可将这个集装箱装上这2艘轮船。如果有，找出一种装载方案。

容易证明，如果一个给定装载问题有解，则采用下面的策略可得到最优装载方案。

(1)首先将第一艘轮船尽可能装满；

(2)将剩余的集装箱装上第二艘轮船。

将第一艘轮船尽可能装满等价于选取全体集装箱的一个子集，使该子集中集装箱重量之和最接近。由此可知，装载问题等价于以下特殊的0-1背包问题。

$$\begin{aligned} \max \quad & \sum_{i=1}^n w_i x_i \\ \text{s.t.} \quad & \sum_{i=1}^n w_i x_i \leq c_1 \\ & x_i \in \{0,1\}, 1 \leq i \leq n \end{aligned}$$

用回溯法设计解装载问题的 $O(2^n)$ 计算时间算法。在某些情况下该算法优于动态规划算法。

装载问题

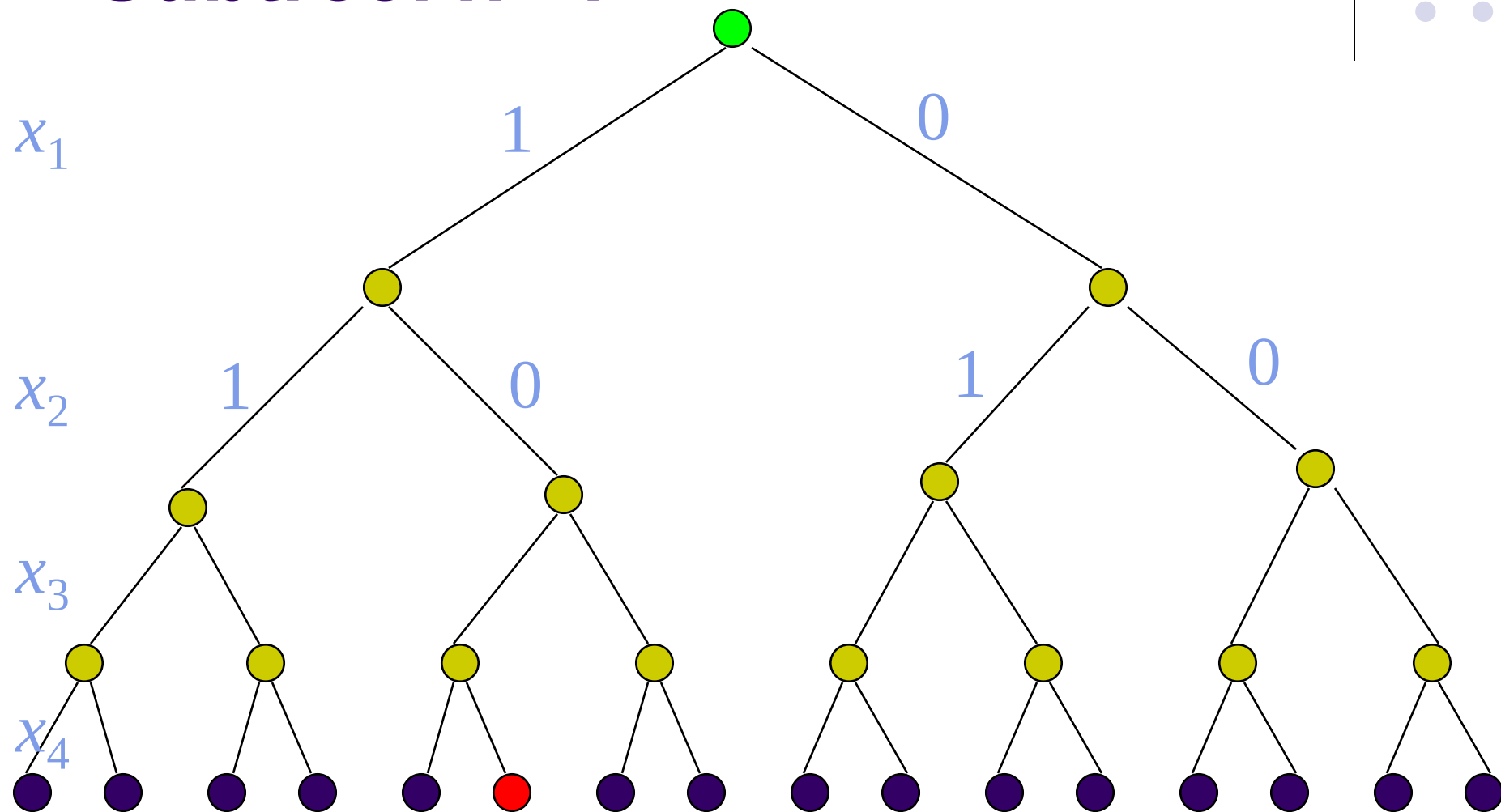
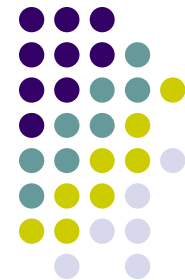


当 $\sum_{i=1}^n w_i = c_1 + c_2$ 时，装载问题等价于子集和(sum-of-subset)问题——有n个数字，要求找到一个子集（如果存在的话）使它的和为c1（属于NP难题）；

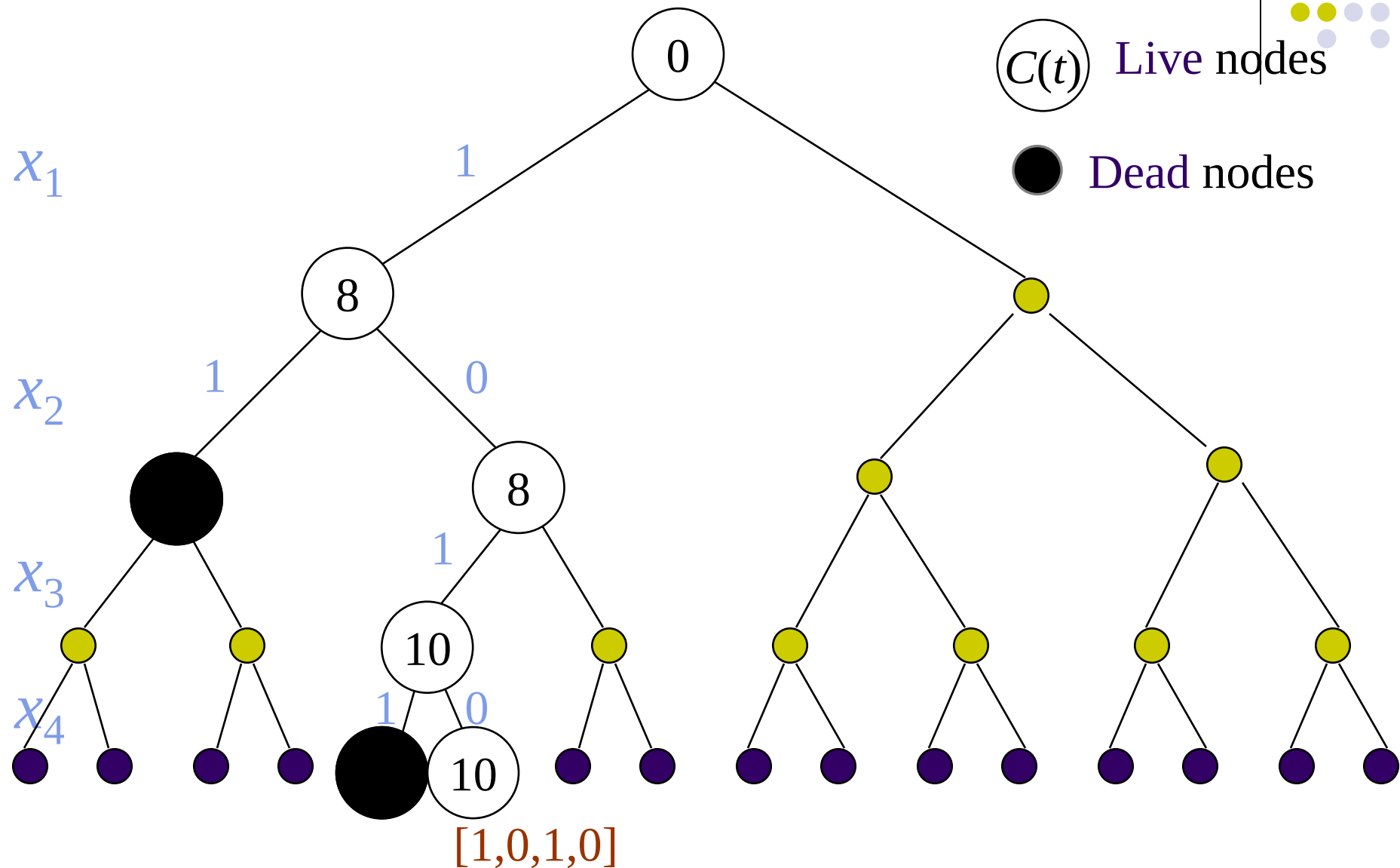
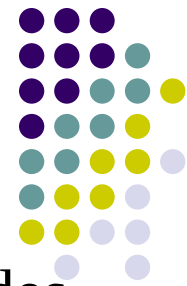
当 $\sum_{i=1}^n w_i = 2c_1$ 时，装载问题等价于划分(partition problem)问题——有n个数字 $a_i, 1 \leq i \leq n$ ，要求找到一个子集（若存在的话），使得子集之和为 $(\sum a_i) / 2$ （也是NP难题）；因此，装载问题是NP难题。

最优装载问题

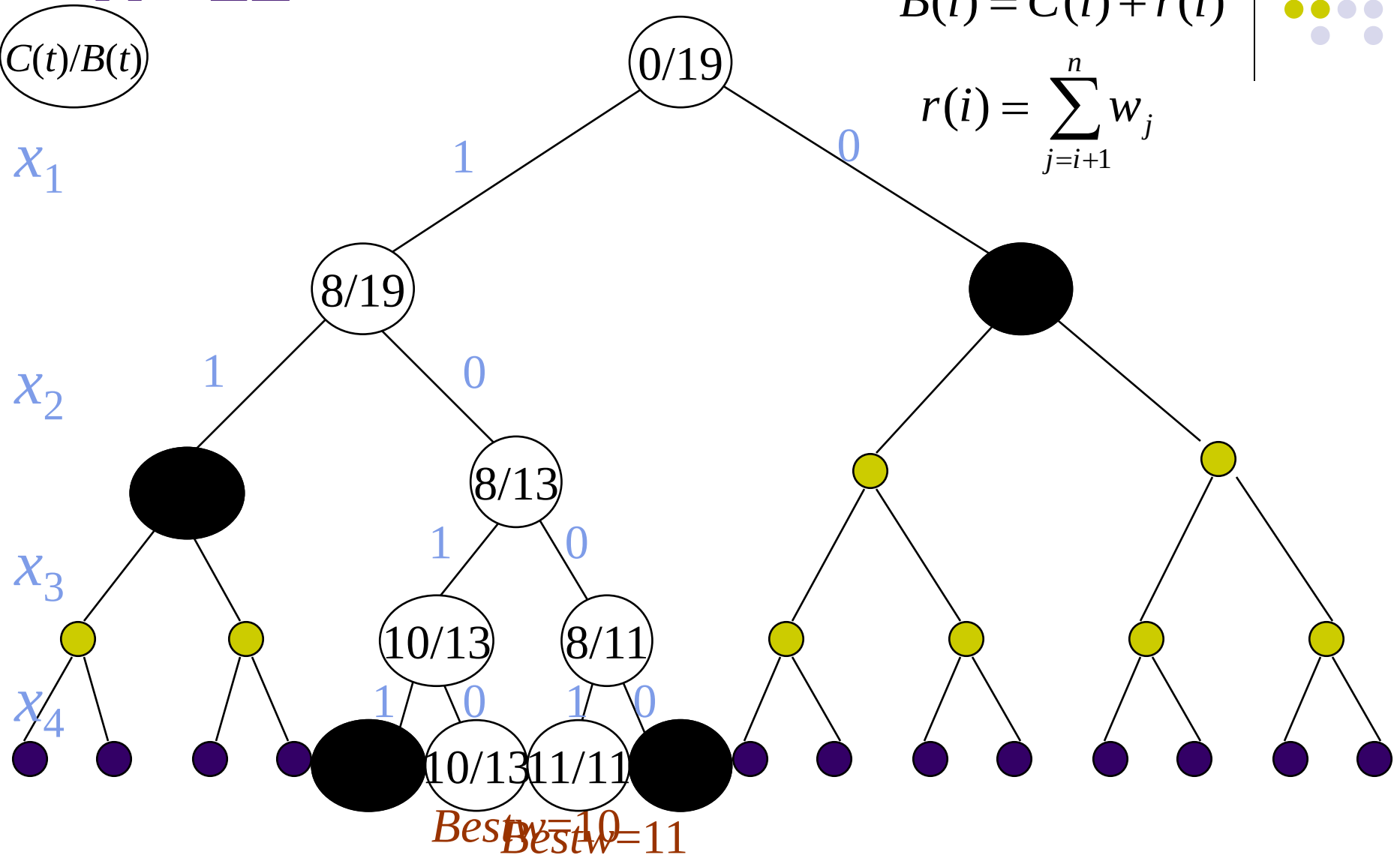
Subtree: $n=4$



Backtracking for $n=4, w=[8,6,2,3], W=12$



$$r(i) = \sum_{j=i+1}^n w_j$$



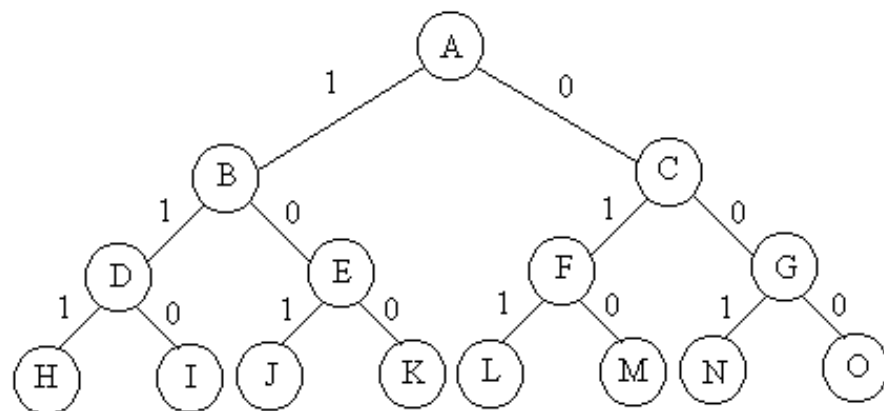
装载问题



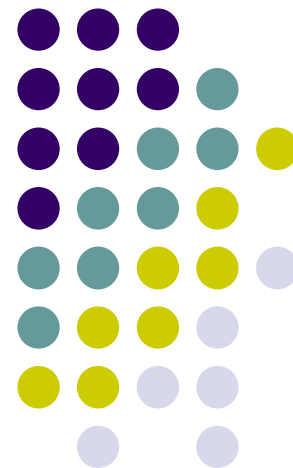
- 解空间：子集树
- 可行性约束函数(选择当前元素): $\sum_{i=1}^n w_i x_i \leq c_1$
- 上界函数(不选择当前元素):

当前载重量 cw +剩余集装箱的重量 $r \leq$ 当前最优载重量 $bestw$

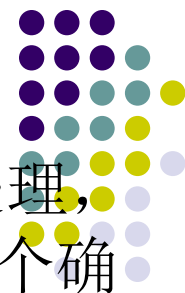
```
void backtrack (int i)
{
    // 搜索第i层结点
    if (i > n) // 到达叶结点
        更新最优解bestx,bestw;return;
    r -= w[i];
    if (cw + w[i] <= c) { // 搜索左子树
        x[i] = 1;
        cw += w[i];
        backtrack(i + 1);
        cw -= w[i];
    }
    if (cw + r > bestw) {
        x[i] = 0; // 搜索右子树
        backtrack(i + 1);
    }
    r += w[i];
}
```



批处理作业调度



批处理作业调度



给定 n 个作业的集合 $\{J_1, J_2, \dots, J_n\}$ 。每个作业必须先由机器1处理，然后由机器2处理。作业 J_i 需要机器 j 的处理时间为 t_{ji} 。对于一个确定的作业调度，设 F_{ji} 是作业 i 在机器 j 上完成处理的时间。所有作业在机器2上完成处理的时间和称为该作业调度的完成时间和。批处理作业调度问题要求对于给定的 n 个作业，制定最佳作业调度方案，使其完成时间和达到最小。

t_{ji}	机器1	机器2
作业1	2	1
作业2	3	1
作业3	2	3

这3个作业的6种可能的调度方案是1,2,3；1,3,2；2,1,3；2,3,1；3,1,2；3,2,1；它们所相应的完成时间和分别是19，18，20，21，19，19。易见，最佳调度方案是1,3,2，其完成时间和为18。

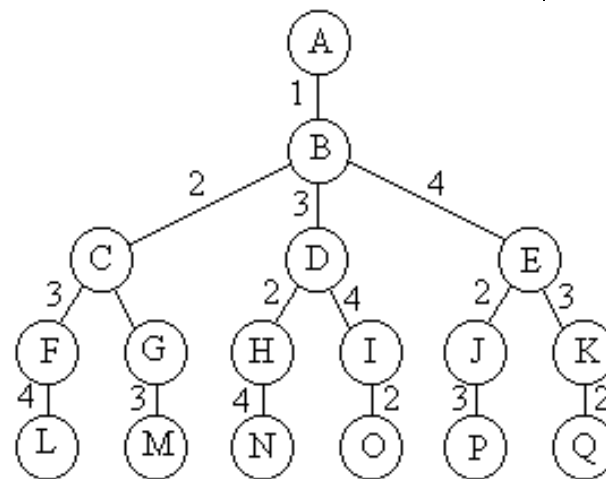
批处理作业调度



•解空间：排列树

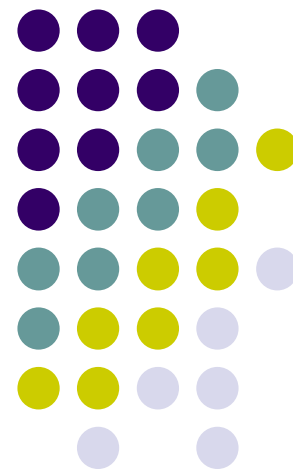
```
void Flowshop::Backtrack(int i)
```

```
{
    if (i > n) {
        for (int j = 1; j <= n; j++)
            bestx[j] = x[j];
        bestf = f;
    }
    else
        for (int j = i; j <= n; j++) {
            f1+=M[x[j]][1];
            f2[i]=((f2[i-1]>f1)?f2[i-1]:f1)+M[x[j]][2];
            f+=f2[i];
            if (f < bestf) {
                Swap(x[i], x[j]);
                Backtrack(i+1);
                Swap(x[i], x[j]);
            }
            f1-=M[x[j]][1];
            f-=f2[i];
        }
}
```



```
class Flowshop {
    friend Flow(int**, int, int []);
private:
    void Backtrack(int i);
    int **M, // 各作业所需的处理时间
        *x, // 当前作业调度
        *bestx, // 当前最优作业调度
        *f2, // 机器2完成处理时间
        f1, // 机器1完成处理时间
        f, // 完成时间和
        bestf, // 当前最优值
        n; // 作业数};
```

符号三角形问题



符号三角形问题



下图是由14个“+”和14个“-”组成的符号三角形。2个同号下面都是“+”，2个异号下面都是“-”。

```

+  +  -  +  -  +  +
  +  -  -  -  -  +
    -  +  +  +  -
      -  +  +  -
        -  +  -
          -  -
            +

```

在一般情况下，符号三角形的第一行有 n 个符号。符号三角形问题要求对于给定的 n ，计算有多少个不同的符号三角形，使其所含的“+”和“-”的个数相同。

符号三角形问题



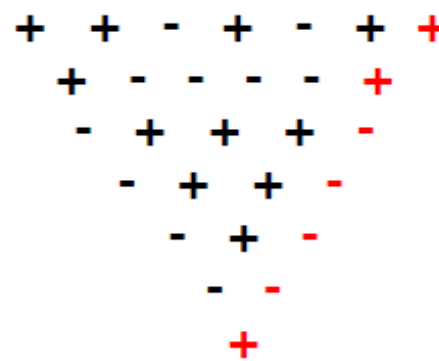
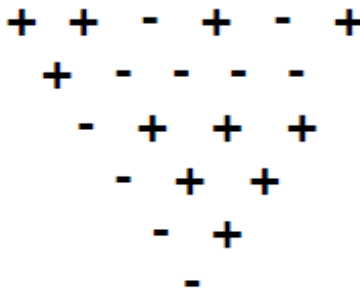
- 解向量：用 n 元组 $x[1:n]$ 表示符号三角形的第一行。
- 可行性约束函数：当前符号三角形所包含的“+”个数与“-”个数均不超过 $n*(n+1)/4$
- 无解的判断： $n*(n+1)/2$ 为奇数

```
void Triangle::Backtrack(int t)
```

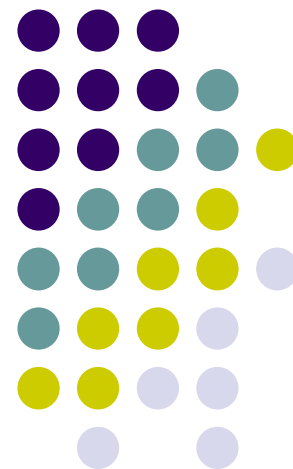
```
{
    if ((count>=n*(n+1)/4 || count<=-n*(n+1)/4)
        return;
    if (t>n) success = true;
    else
        for (int i=1; i<=n-t+1; i++) {
            p[1][t]=p[i][t-i+1];
            count+=p[i][t-i+1];
            for (int j=2; j<=t; j++) {
                p[j][t-j+1]=p[j-1][t-j+1]^p[j-1][t-j+2];
                count+=p[j][t-j+1];
            }
            Backtrack(t+1);
            for (int j=2; j<=t; j++)
                count-=p[j][t-j+1];
            count-=i;
        }
}
```

复杂度分析

计算可行性约束需要 $O(n)$ 时间，在最坏情况下有 $O(2^n)$ 个结点需要计算可行性约束，故解符号三角形问题的回溯算法所需的计算时间为 $O(n2^n)$ 。



n后问题



n后问题

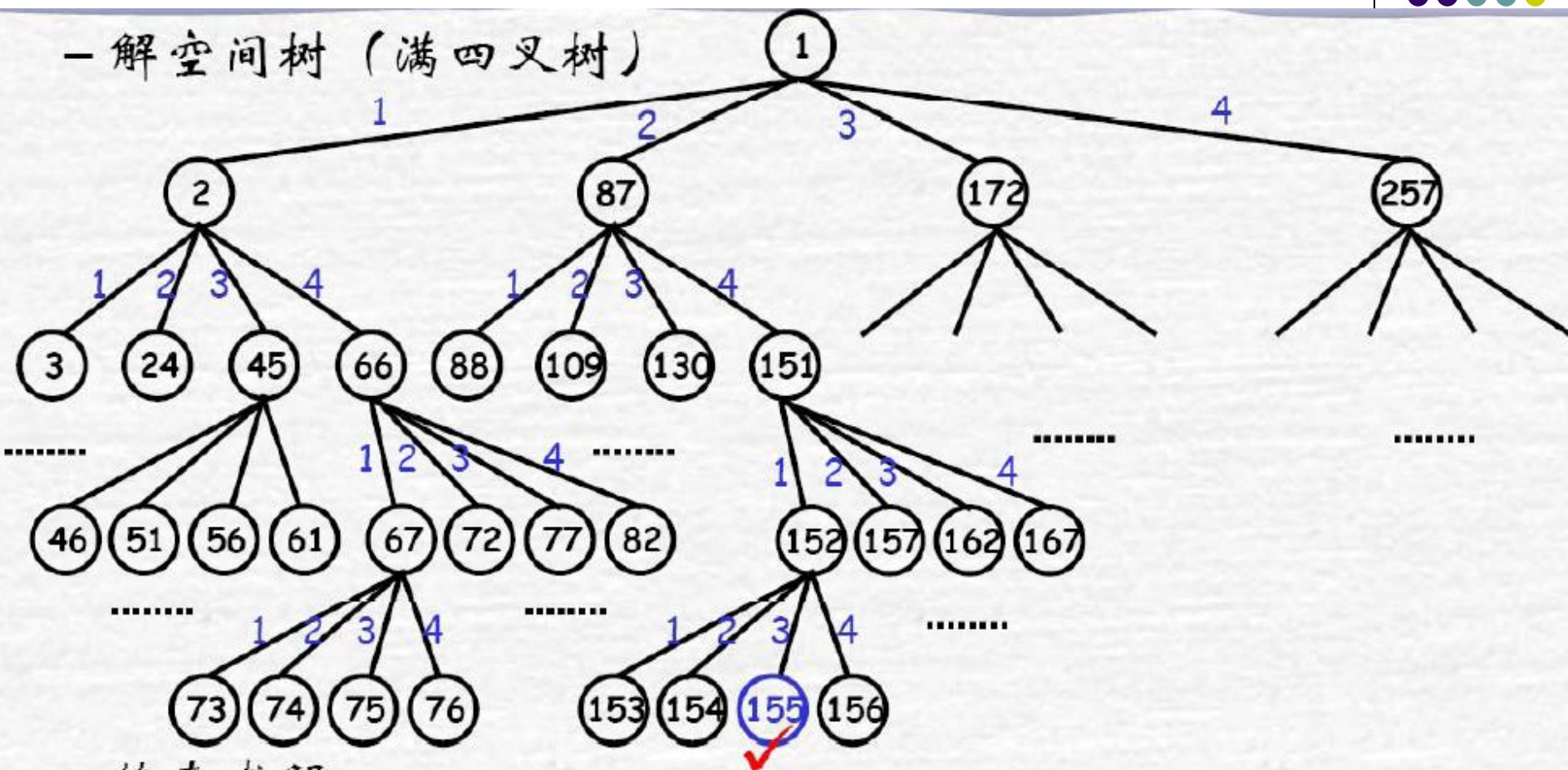


在 $n \times n$ 格的棋盘上放置彼此不受攻击的 n 个皇后。按照国际象棋的规则，皇后可以攻击与之处在同一行或同一列或同一斜线上的棋子。 n 后问题等价于在 $n \times n$ 格的棋盘上放置 n 个皇后，任何2个皇后不放在同一行或同一列或同一斜线上。

1				Q				
2						Q		
3								Q
4		Q						
5							Q	
6	Q							
7			Q					
8					Q			
	1	2	3	4	5	6	7	8



一 解空间树 (满四叉树)



一 搜索求解:

从①起按DFS搜索, 搜索时应满足隐约束, 搜索到叶结点输出解: (2,4,1,3), (3,1,4,2)

n后问题

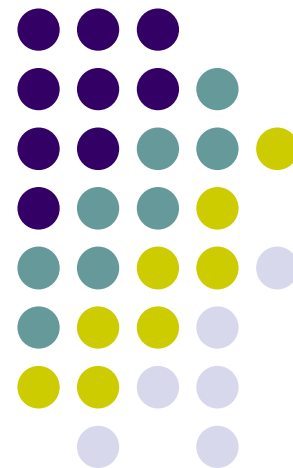


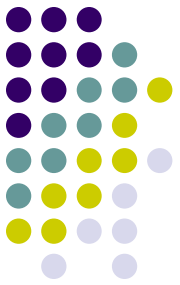
- 解向量: $(x_1, x_2, \dots, x_n)$
- 显约束: $x_i=1, 2, \dots, n$
- 隐约束:
 - 1)不同列: $x_i \neq x_j$
 - 2)不处于同一正、反对角线: $|i-j| \neq |x_i - x_j|$

```
bool Queen::Place(int k)
{
    for (int j=1; j<k; j++)
        if ((abs(k-j)==abs(x[j]-x[k])) || (x[j]==x[k])) return false;
    return true;
}
```

```
void Queen::Backtrack(int t)
{
    if (t>n) sum++;
    else
        for (int i=1; i<=n; i++) {
            x[t]=i;
            if (Place(t)) Backtrack(t+1);
        }
}
```

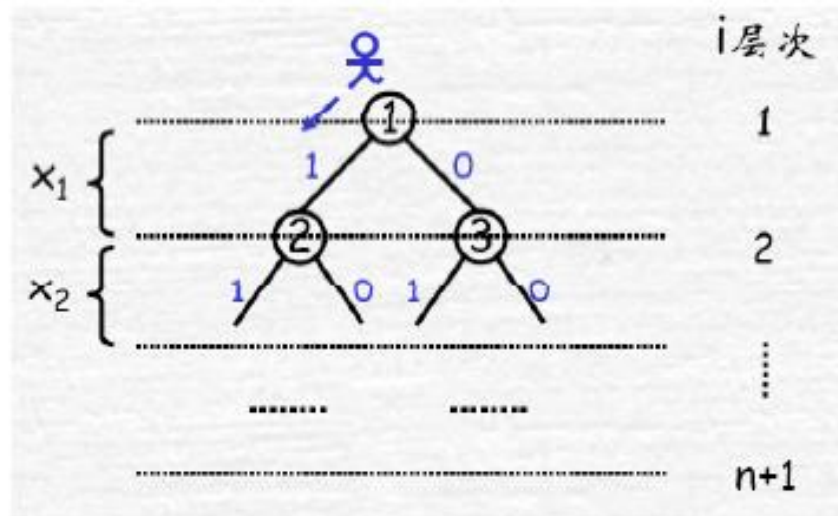

0-1背包问题



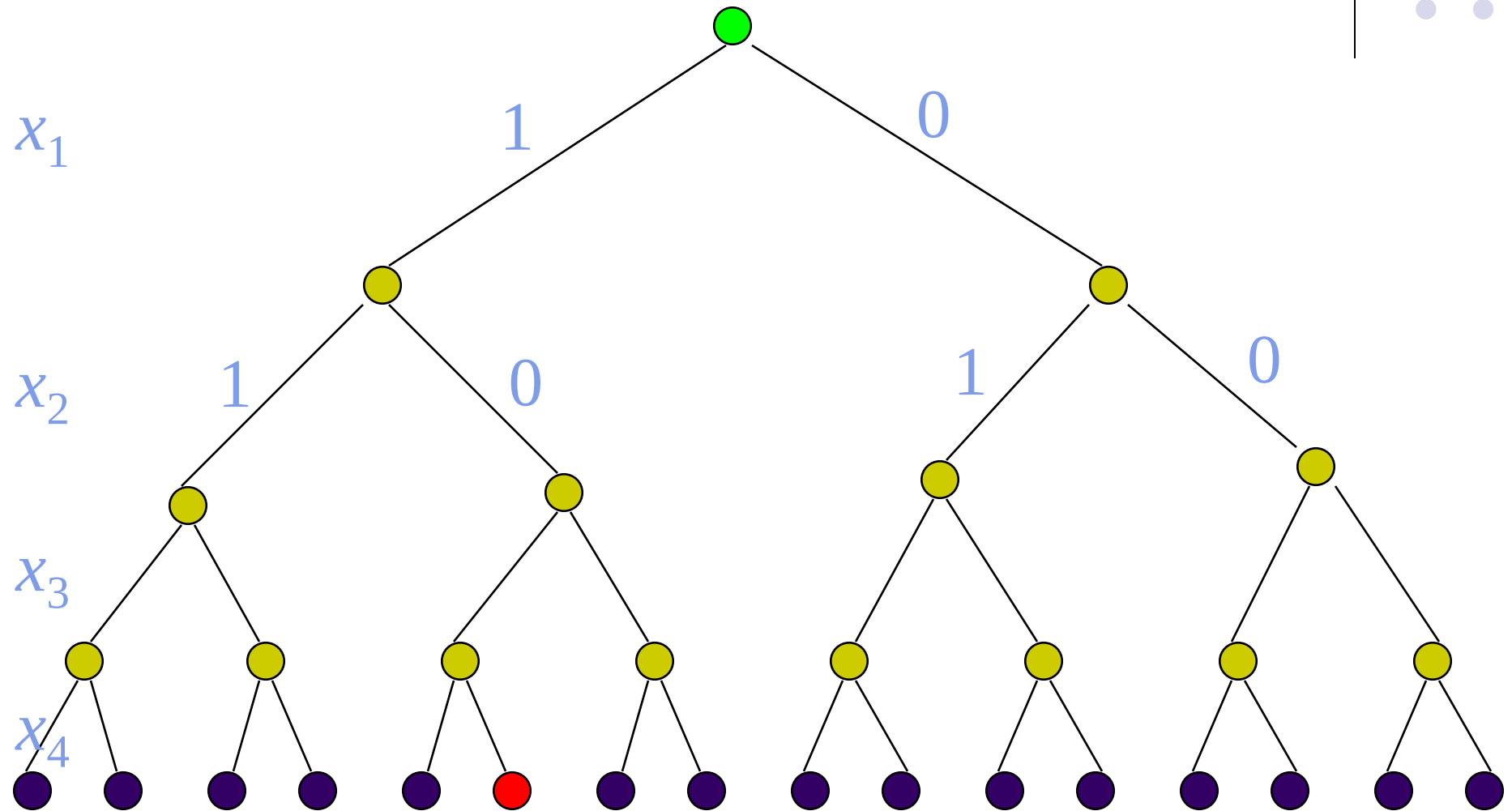
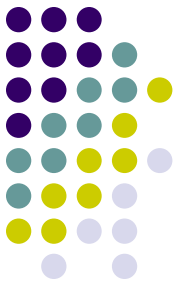


0-1背包问题

- 问题描述：略
- 解表示和解空间： $\{ (x_1, x_2, \dots, x_n) \mid x_i \in \{0, 1\}, i=1 \sim n \}$
- 解空间树：



Subtree: $n=4$





0-1背包问题

- 无限界函数的算法

```
KnapBacktrack(int i)
```

```
{ //cw当前背包重量, cv当前背包价值, bestv当前最优价值
```

```
    if(i > n) { //搜索到可行解
```

```
        bestv = (bestv < cv) ? cv : bestv;
```

```
        output(x);
```

```
    }
```

```
    else{
```

```
        if ( cw + w[i] <= c ) { //走左子树
```

```
            x[i] = 1; cw += w[i]; cv += v[i];
```

```
            KnapBacktrack(i+1);
```

```
            cw -= w[i]; cv -= v[i];
```

```
        }
```

```
        //以下走右子树
```

```
        x[i] = 0;
```

```
        KnapBacktrack(i+1);
```

```
    }
```

```
main(float c, int n, float w[], float v[], int x[])
```

```
{ //主程序
```

```
    float cw=0.0, cv = 0.0, bestv = 0.0;
```

```
    KnapBacktrack(1);
```

```
}
```



0-1背包问题

● 有限界函数的算法

— 基本思想：

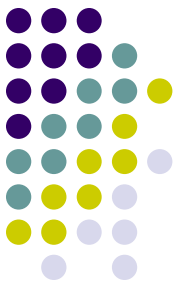
- 设 r 是当前扩展结点 Z 的右子树（或左子树）的价值上界，如果 $cv+r \leq bestv$ 时，则可以裁剪掉右子树（或左子树）。
- 一种简单的确定 Z 的左、右子树最优值上界的方法（设 Z 为第 k 层结点）：
左子树上界 = $\sum_{i=k}^n v_i$ ，右子树上界 = $\sum_{i=k+1}^n v_i$

— 求经扩展结点 Z 的可行解价值上界的方法：

- 计算至扩展结点的当前背包价值
已知 x_i , $i=1 \sim k-1$, 当前背包价值 $cv = \sum_{i=1}^{k-1} v_i x_i$
- 最后，
经 Z 左子树的可行解价值上界 = $cv + \text{左子树上界}$
经 Z 右子树的可行解价值上界 = $cv + \text{右子树上界}$

— 算法（略）

$n=4, W=7, v=[4,7,9,10], w=[1,2,3,5]$
 $v/w=[4, 3.5, 3, 2]$.

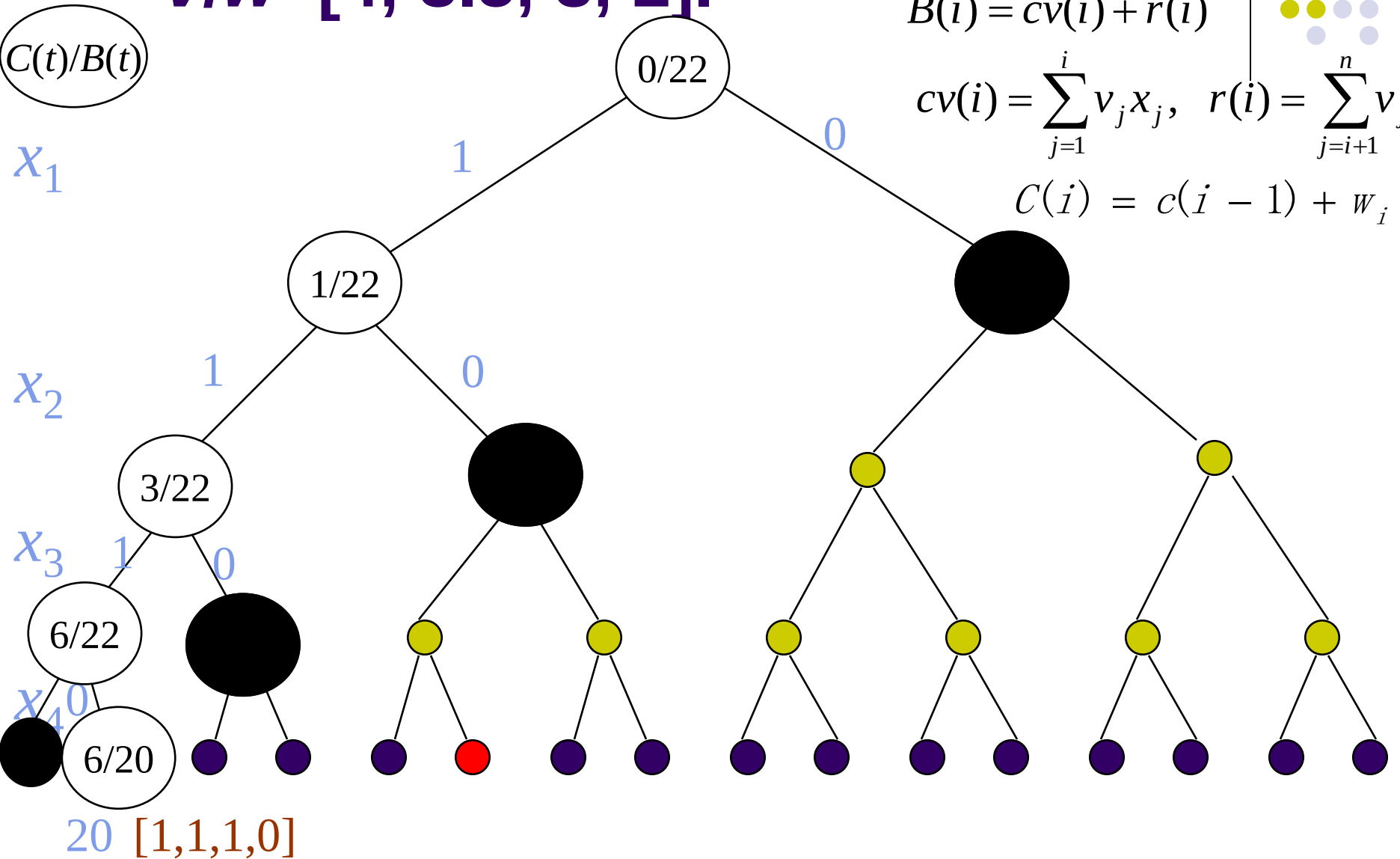


$C(t)/B(t)$

$$B(i) = cv(i) + r(i)$$

$$cv(i) = \sum_{j=1}^i v_j x_j, \quad r(i) = \sum_{j=i+1}^n v_j$$

$$C(i) = C(i-1) + w_i$$



0-1背包问题



- 解空间：子集树
- 可行性约束函数： $\sum_{i=1}^n w_i x_i \leq c_1$
- 上界函数：

```
template<class Typew, class Typep>
```

```
Typep Knap<Typew, Typep>::Bound(int i)
```

```
{// 计算上界
```

```
    Typew cleft = c - cw; // 剩余容量
```

```
    Typep b = cp;
```

```
    // 以物品单位重量价值递减序装入物品
```

```
    while (i <= n && w[i] <= cleft) {
```

```
        cleft -= w[i];
```

```
        b += p[i];
```

```
        i++;
```

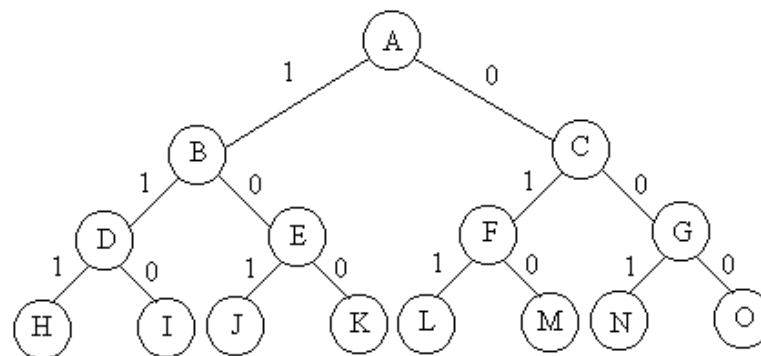
```
    }
```

```
    // 装满背包
```

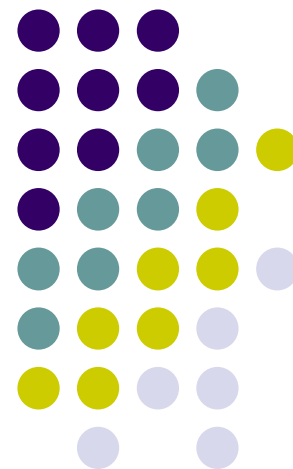
```
    if (i <= n) b += p[i]/w[i] * cleft;
```

```
    return b;
```

```
}
```



最大团问题



最大团问题

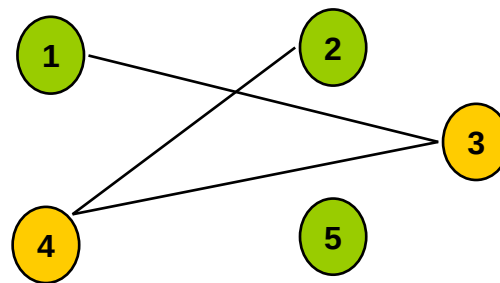
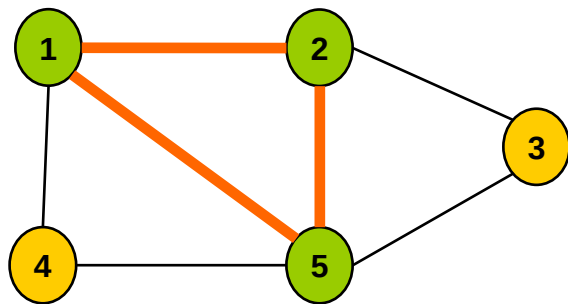


给定无向图 $G=(V, E)$ 。如果 $U \subseteq V$ ，且对任意 $u, v \in U$ 有 $(u, v) \in E$ ，则称 U 是 G 的完全子图。 G 的完全子图 U 是 G 的团当且仅当 U 不包含在 G 的更大的完全子图中。 G 的最大团是指 G 中所含顶点数最多的团。

如果 $U \subseteq V$ 且对任意 $u, v \in U$ 有 $(u, v) \notin E$ ，则称 U 是 G 的空子图。 G 的空子图 U 是 G 的独立集当且仅当 U 不包含在 G 的更大的空子图中。 G 的最大独立集是 G 中所含顶点数最多的独立集。

对于任一无向图 $G=(V, E)$ 其补图 $\bar{G}=(V, E_1)$ 定义为： $V_1=V$ ，且 $(u, v) \in E_1$ 当且仅当 $(u, v) \notin E$ 。

U 是 G 的最大团当且仅当 U 是 $\bar{G}$ 的最大独立集。



最大团问题



- 解空间：子集树
- 可行性约束函数：顶点*i*到已选入的顶点集中每一个顶点都有边相连。
- 上界函数：有足够多的可选择顶点使得算法有可能在右子树中找到更大的团。

```
void Clique::Backtrack(int i)
```

```
{// 计算最大团
```

```
  if (i > n) {// 到达叶结点
```

```
    for (int j = 1; j <= n; j++) bestx[j] = x[j];
```

```
    bestn = cn; return;}
```

```
  // 检查顶点 i 与当前团的连接
```

```
  int OK
```

```
  for (int
```

```
    if (x[j]
```

```
    // i
```

```
    OK
```

```
  if (OK) {// 进入左子树
```

```
    x[i] = 1; cn++;
```

```
    Backtrack(i+1);
```

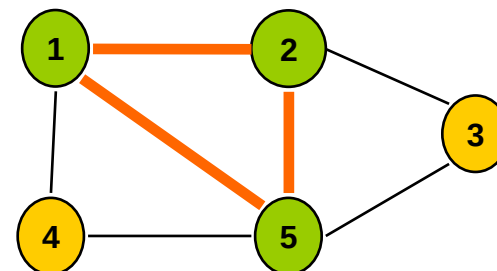
```
    x[i] = 0; cn--;} 
```

```
  if (cn + n - i > bestn) {// 进入右子树
```

```
    x[i] = 0;
```

```
    Backtrack(i+1);} 
```

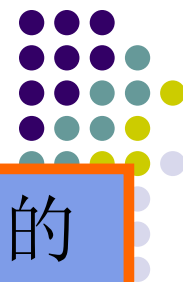
```
}
```



复杂度分析

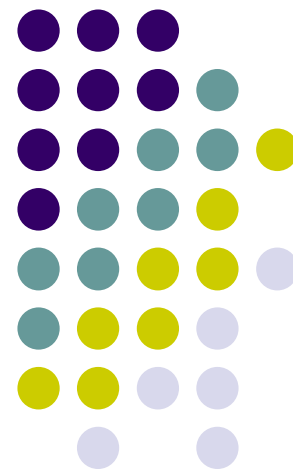
最大团问题的回溯算法**backtrack**所需的计算时间显然为 $O(n2^n)$ 。

进一步改进



- 选择合适的搜索顺序，可以使得上界函数更有效的发挥作用。例如在搜索之前可以将顶点按度从小到大排序。这在某种意义上相当于给回溯法加入了启发性。
- 定义 $S_i = \{v_i, v_{i+1}, \dots, v_n\}$ ，依次求出 $S_n, S_{n-1}, \dots, S_1$ 的解。从而得到一个更精确的上界函数，若 $cn + S_i \leq \max$ 则剪枝。同时注意到：从 S_{i+1} 到 S_i ，如果找到一个更大的团，那么 v_i 必然属于找到的团，此时有 $S_i = S_{i+1} + 1$ ，否则 $S_i = S_{i+1}$ 。因此只要 $\max$ 的值被更新过，就可以确定已经找到最大值，不必再往下搜索了。

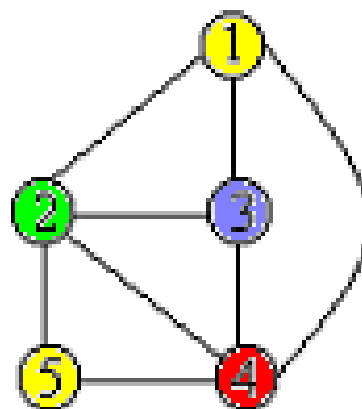
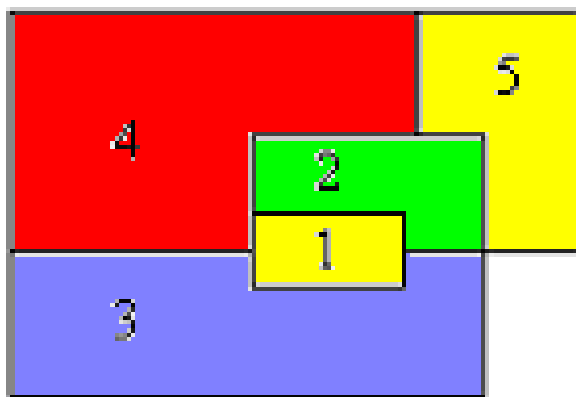
图的m着色问题

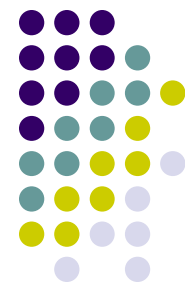


图的m着色问题



给定无向连通图 G 和 m 种不同的颜色。用这些颜色为图 G 的各顶点着色，每个顶点着一种颜色。是否有一种着色法使 G 中每条边的2个顶点着不同颜色。这个问题是图的 m 可着色判定问题。若一个图最少需要 m 种颜色才能使图中每条边连接的2个顶点着不同颜色，则称这个数 m 为该图的色数。求一个图的色数 m 的问题称为图的 m 可着色优化问题。





四色原理

- 世界近代三大数学难题之一(另外两个是费马定理和哥德巴赫猜想)。
- 这是一个拓扑学问题，即找出给球面（或平面）地图着色时所需用的不同颜色的最小数目。着色时要使得没有两个相邻（即有公共边界线段）的区域有相同的颜色。**1852**年英国的格思里推测：四种颜色是充分必要的。**1878**年英国数学家凯利在一次数学家会议上呼吁大家注意解决这个问题。直到**1976**年，美国数学家阿佩哈尔、哈肯和考西利用高速电子计算机运算了**1200**个小时，才证明了格思里的推测。四色问题的解决在数学研究方法上的突破，开辟了机器证明的美好前景。
- 四色定理是第一个主要由计算机证明的理论，证明方法将地图上的无限种可能情况减少为**1,936**种状态（稍后减少为**1,476**种），这些状态由计算机一个挨一个的进行检查。

图的m着色问题

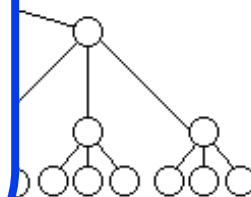


- 解向量: $(x_1, x_2, \dots, x_n)$ 表示顶点*i*所着颜色 $x[i]$
- 可行性约束函数: 顶点*i*与已着色的相邻顶点颜色不重复。

复杂度分析

图m可着色问题的解空间树中内结点个数是 $\sum_{i=0}^{n-1} m^i$
对于每一个内结点, 在最坏情况下, 用ok检查当前扩展结点的每一个儿子所相应的颜色可用性需耗时 $O(mn)$ 。因此, 回溯法总的时间耗费是

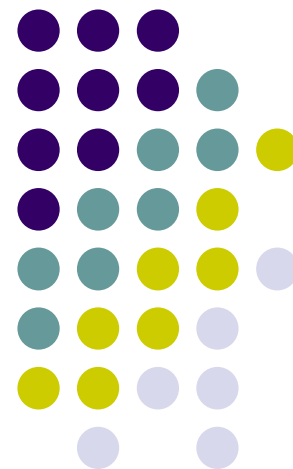
$$\sum_{i=0}^{n-1} m^i (mn) = nm(m^n - 1) / (m - 1) = O(nm^n)$$



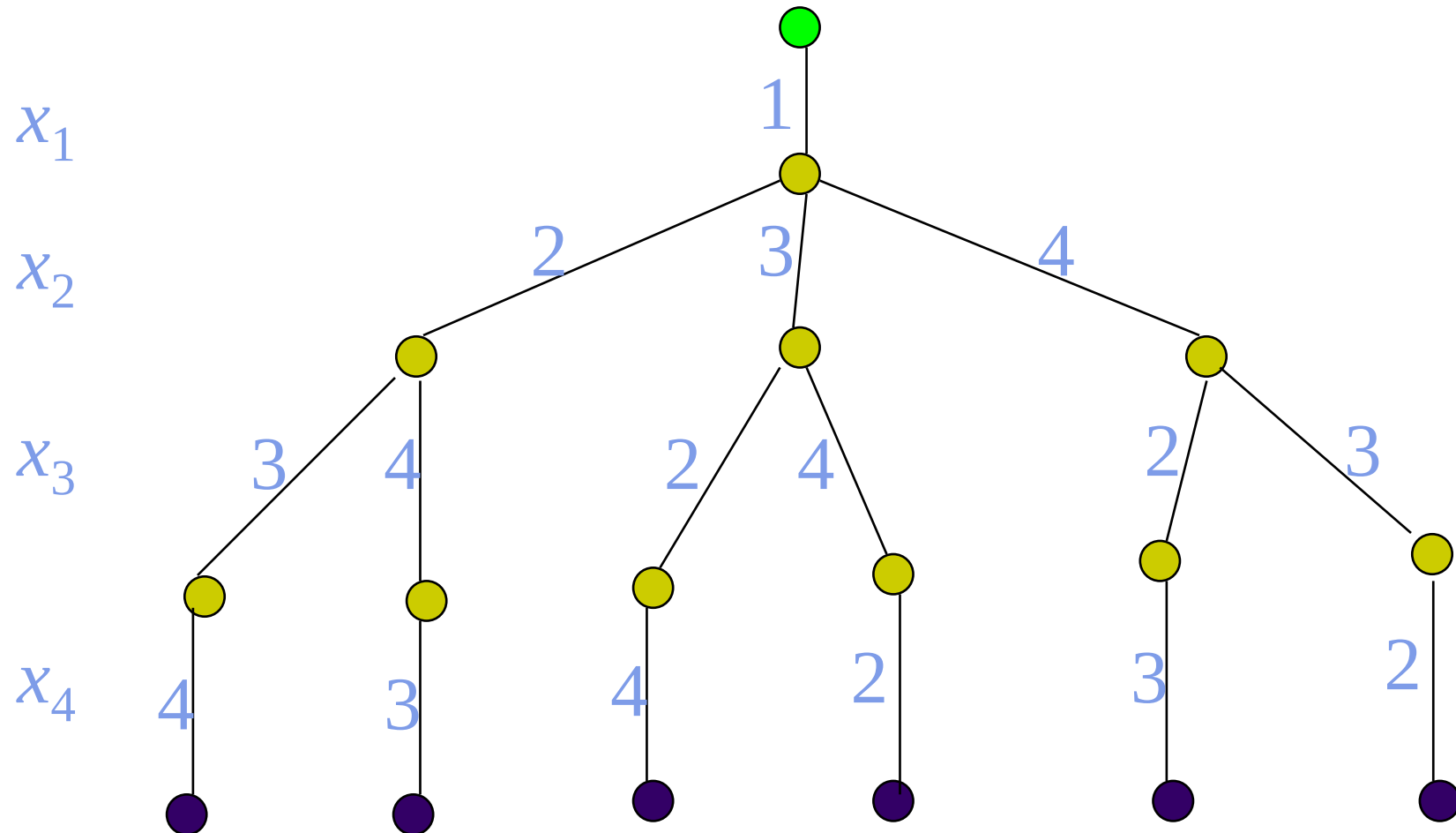
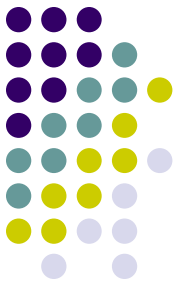
```
void Backtrack(int t)
{
    if (t > n)
        success = true;
    for (int c = 1; c <= m; c++)
    {
        if (Color::Ok(t, c))
        {
            x[t] = c;
            if (Backtrack(t+1))
                return;
        }
    }
}

bool Color::Ok(int k)
{
    // 检查颜色可用性
    for (int j=1; j<=n; j++)
        if ((a[k][j]==1)&&(x[j]==x[k])) return false;
    return true;
}
```

旅行售货员问题



Permutation tree: $n=4$



旅行售货员问题



- 解空间：排列树

```
template<class Type>
void Traveling<Type>::Backtrack(int i)
{
    if (i == n) {
```

复杂度分析

算法**backtrack**在最坏情况下可能需要更新当前最优解 $O((n-1)!)$ 次，每次更新**bestx**需计算时间 $O(n)$ ，从而整个算法的计算时间复杂度为 $O(n!)$ 。

```
        // 是否可进入x[j]子树？
        if (a[x[i-1]][x[j]] != NoEdge &&
            (cc + a[x[i-1]][x[j]] < bestc || bestc == NoEdge)) {
            // 搜索子树
            Swap(x[i], x[j]);
            cc += a[x[i-1]][x[j]];
            Backtrack(i+1);
            cc -= a[x[i-1]][x[j]];
            Swap(x[i], x[j]);
        }
    }
```

Backtracking

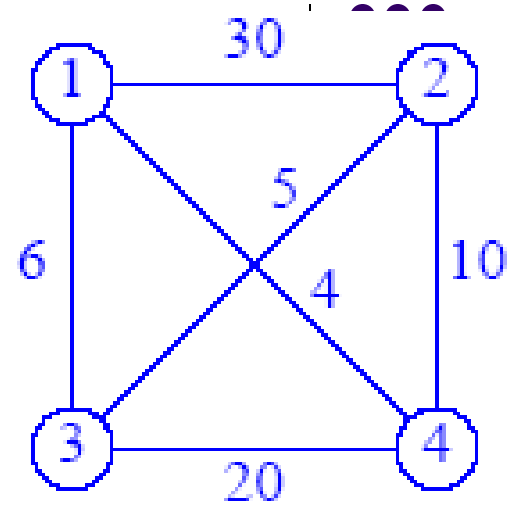
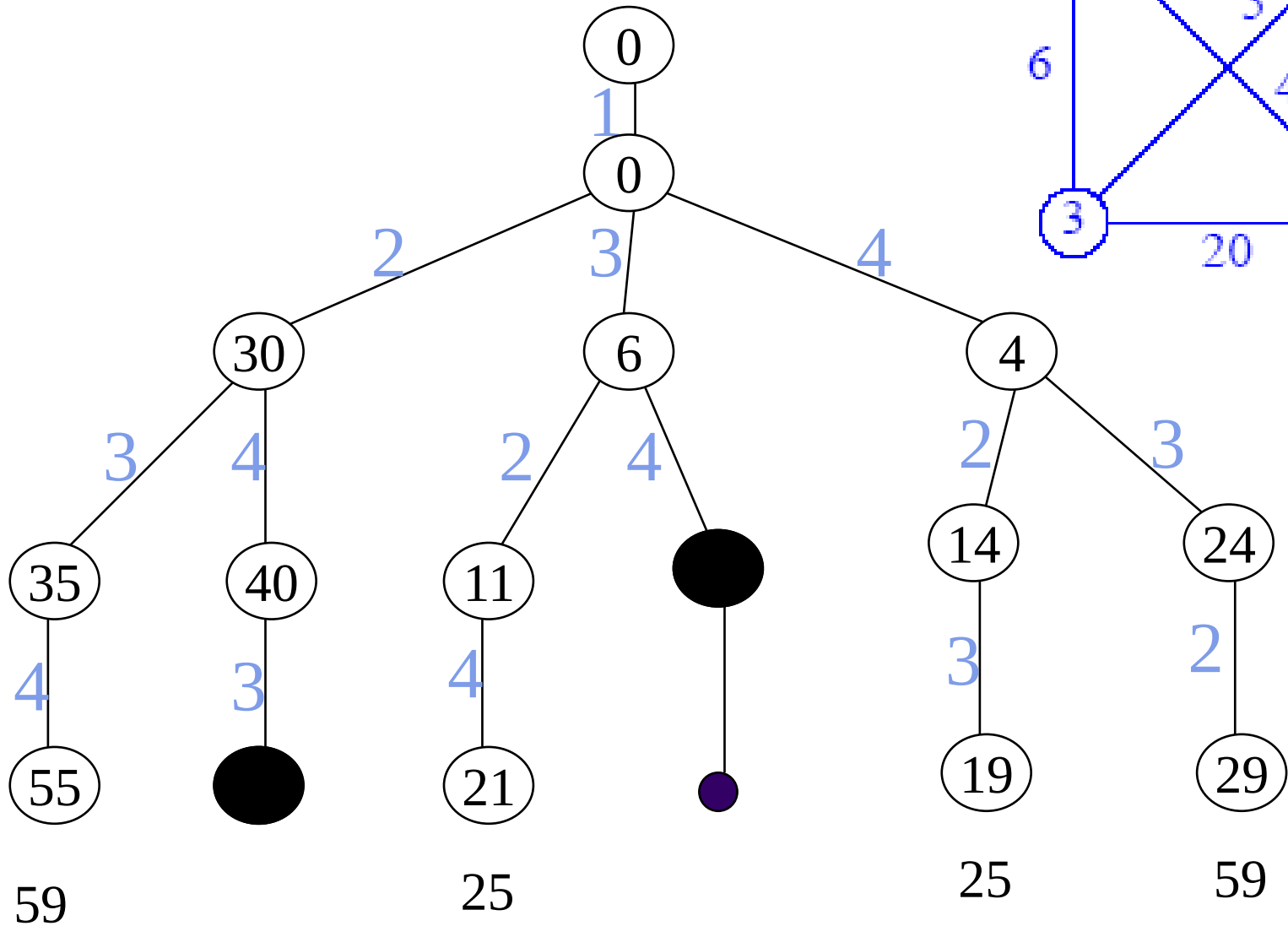
B(i)

x_1

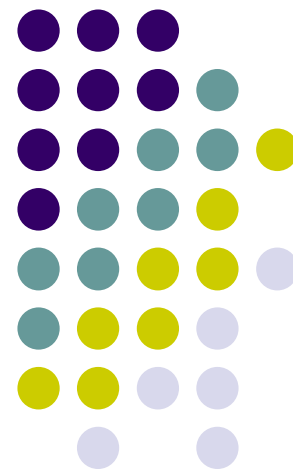
x_2

x_3

x_4



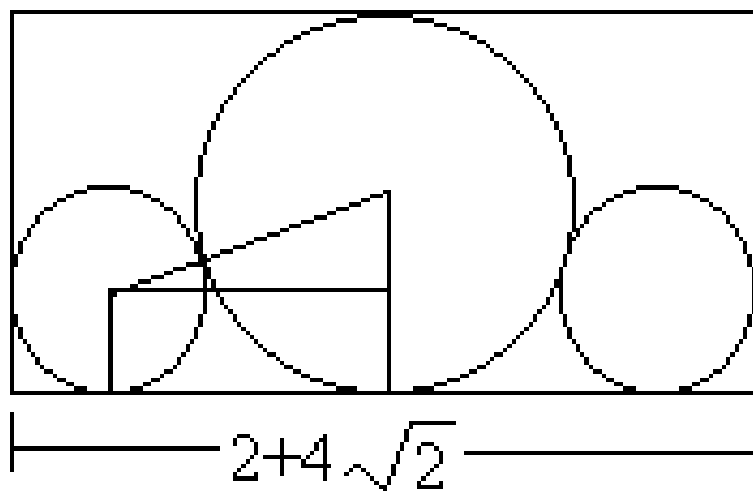
圆排列问题



圆排列问题



给定 n 个大小不等的圆 $c_1, c_2, \dots, c_n$ ，现要将这 n 个圆排进一个矩形框中，且要求各圆与矩形框的底边相切。圆排列问题要求从 n 个圆的所有排列中找出有最小长度的圆排列。例如，当 $n=3$ ，且所给的3个圆的半径分别为1, 1, 2时，这3个圆的最小长度的圆排列如图所示。其最小长度为 $2 + 4\sqrt{2}$



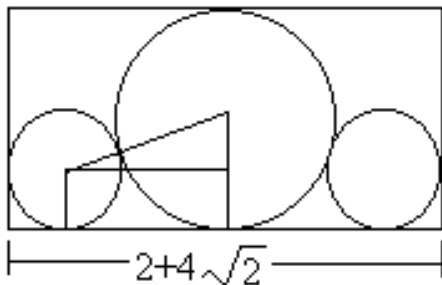
圆排列问题

```
void Circle::Backtrack(int t)
{
    if (t>n) Compute();
    else
        for (int i = t; i <= n; i++) {
```

```
float Circle::Center(int t)
{// 计算当前所选择圆的圆心横坐标
    float temp=0;
    for (int j=1;j<t;j++) {
```

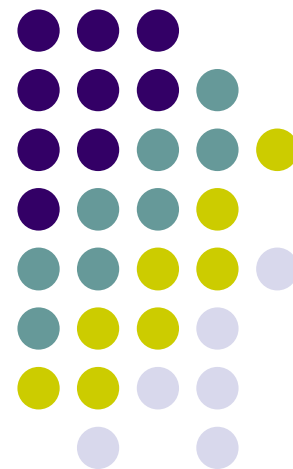
- 上述算法尚有许多改进的余地。例如，象 $1,2,\dots,n-1,n$ 和 $n,n-1,\dots,2,1$ 这种互为镜像的排列具有相同的圆排列长度，只计算一个就够了，可减少约一半的计算量。另一方面，如果所给的 n 个圆中有 k 个圆有相同的半径，则这 k 个圆产生的 $k!$ 个完全相同的圆排列，只计算一个就够了。

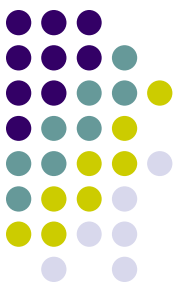
```
}
```



```
// 计算并返回圆排列的长度
float low=0,
      high=0;
for (int i=1;i<=n;i++) {
    if (x[i]-r[i]<low) low=x[i]-r[i];
    if (x[i]+r[i]>high) high=x[i]+r[i];
}
if (high-low<min) min=high-low;
}
```

电路板排列问题

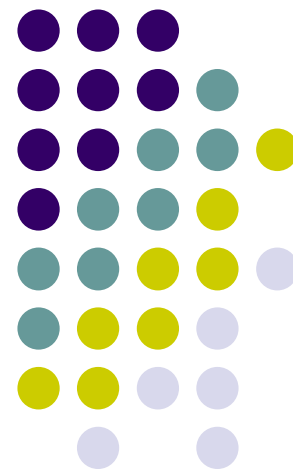




电路板排列问题

- n 个电路板的每一种排列定义了一种放置方法。令 $B = \{b_1, \dots, b_n\}$ 表示这 n 个电路板。 m 个网组集合 $L = \{N_1, \dots, N_m\}$ 由电路板定义， N_i 是 B 的子集，子集中的元素需要连接起来。实际中用电线将插有这些电路板的插槽连接起来。
- 例 令 $n=8$, $m=5$ 。集合 B 和 L 如下：
 $B = \{b_1, b_2, b_3, b_4, b_5, b_6, b_7, b_8\}$
 $L = \{N_1, N_2, N_3, N_4, N_5\}$
 $N_1 = \{b_4, b_5, b_6\}$
 $N_2 = \{b_2, b_3\}$
 $N_3 = \{b_1, b_3\}$
 $N_4 = \{b_3, b_6\}$
 $N_5 = \{b_7, b_8\}$
- 令 x 为电路板的一个排列。电路板 x_i 被放置到机箱的插槽 i 中。
 $\text{density}(x)$ 为机箱中任意一对相邻插槽间所连电线数目中的最大值
- 电路板排列问题的目标是找到一种电路板的排列方式，使其有最小的 density 。

连续邮资问题



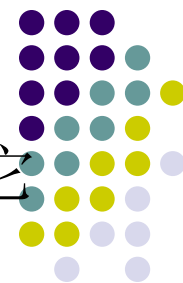
连续邮资问题



假设国家发行了 n 种不同面值的邮票，并且规定每张信封上最多只允许贴 m 张邮票。连续邮资问题要求对于给定的 n 和 m 的值，给出邮票面值的最佳设计，在1张信封上可贴出从邮资1开始，增量为1的最大连续邮资区间。

例如，当 $n=5$ 和 $m=4$ 时，面值为(1,3,11,15,32)的5种邮票可以贴出邮资的最大连续邮资区间是1到70。

连续邮资问题



- 解向量：用 n 元组 $x[1:n]$ 表示 n 种不同的邮票面值，并约定它们从小到大排列。 $x[1]=1$ 是唯一的选择。
- 可行性约束函数：已选定 $x[1:i-1]$ ，最大连续邮资区间是 $[1:r]$ ，接下来 $x[i]$ 的可取值范围是 $[x[i-1]+1:r+1]$ 。

如何确定 r 的值？

计算 $X[1:i]$ 的最大连续邮资区间在本算法中被频繁使用到，因此势必要找到一个高效的方法。考虑到直接递归的求解复杂度太高，我们不妨尝试计算用不超过 m 张面值为 $x[1:i]$ 的邮票贴出邮资 k 所需的最少邮票数 $y[k]$ 。通过 $y[k]$ 可以很快推出 r 的值。事实上， $y[k]$ 可以通过递推在 $O(n)$ 时间内解决：

```
for (int j=0; j<= x[i-2]*(m-1);j++)
    if (y[j]<m)
        for (int k=1;k<=m-y[j];k++)
            if (y[j]+k<y[j+x[i-1]*k]) y[j+x[i-1]*k]=y[j]+k;
while (y[r]<maxint) r++;
```

回溯法效率分析



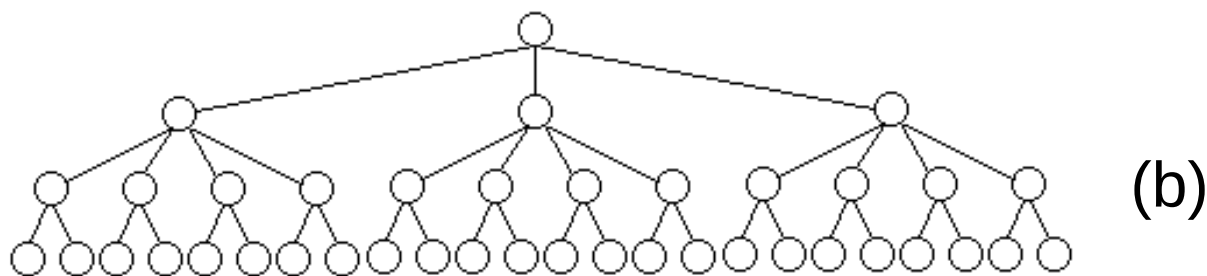
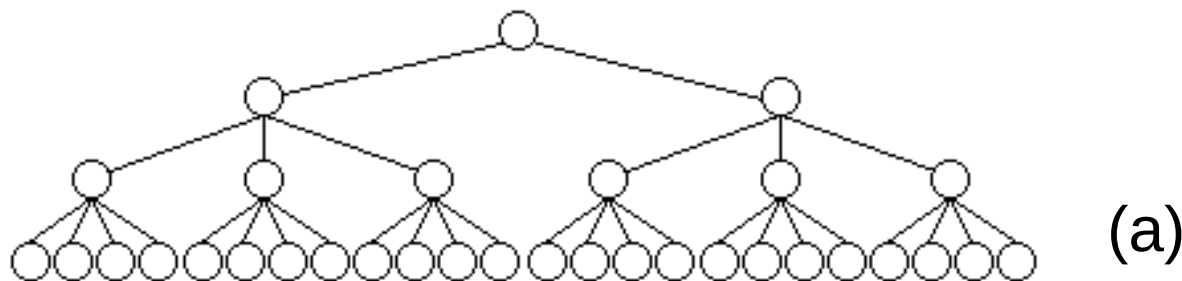
通过前面具体实例的讨论容易看出，回溯算法的效率在很大程度上依赖于以下因素：

- (1)产生 $x[k]$ 的时间；
 - (2)满足显约束的 $x[k]$ 值的个数；
 - (3)计算约束函数**constraint**的时间；
 - (4)计算上界函数**bound**的时间；
 - (5)满足约束函数和上界函数约束的所有 $x[k]$ 的个数。
- 好的约束函数能显著地减少所生成的结点数。但这样的约束函数往往计算量较大。因此，在选择约束函数时通常存在生成结点数与约束函数计算量之间的折衷。

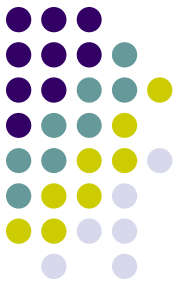
重排原理



对于许多问题而言，在搜索试探时选取 $x[i]$ 的值顺序是任意的。
在其它条件相当的前提下，让可取值最少的 $x[i]$ 优先。从图中关于同一问题的2棵不同解空间树，可以体会到这种策略的潜力。



图(a)中，从第1层剪去1棵子树，则从所有应当考虑的3元组中一次消去12个3元组。对于图(b)，虽然同样从第1层剪去1棵子树，却只从应当考虑的3元组中消去8个3元组。前者的效果明显比后者好。



本周任务

- 课后习题5-1, 5-2, 5-7, 5-16, 5-25
- 附加题
 - 证明放在 x_i 和 x_j 位置上的两个皇后在同一对角线上当且仅当
 - $x_i - x_j = i - j$ 或 $x_i - x_j = j - i$
 - 设计一个回溯算法求解马周游问题：给出一个 8×8 的棋盘，一个放在棋盘某个位置上的马是否可以恰好访问每个方格一次，并回到起始位置上？